

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

ПРАКТИЧНА РОБОТА № 6

З дисципліни «Інженерія програмного забезпечення»

на тему «ПОРОДЖУВАЛЬНІ ШАБЛони. ШАБЛони PROTOTYPE,
SINGLETON, FACTORY METHOD, ABSTRACT FACTORY, BUILDER.»

Виконав:
студент групи ІО-42
Куліков М. М.
Залікова: 4214

Перевірила:
Ст. викладач кафедри ОТ
Васильєва М. Д.

Практична робота №6

Тема: «ПОРОДЖУВАЛЬНІ ШАБЛОНИ. ШАБЛОНИ PROTOTYPE, SINGLETON, FACTORY METHOD, ABSTRACT FACTORY, BUILDER».

Мета: Вивчення шаблонів поведінки. Отримання базових навичок з застосування шаблонів Prototype, Singleton, Factory Method, Abstract Factory та Builder.

Виконання роботи:

Завдання 1

1. Повторити породжувальні шаблони проектування ПЗ. Знати загальну характеристику породжувальних шаблонів та призначення кожного з них.
2. Детально вивчити породжувальні шаблони для проектування ПЗ – Prototype, Singleton, Factory Method Abstract Factory та Builder. Для кожного з них:
 - вивчити Шаблон, його призначення, мотивацію, випадки коли його застосування є доцільним та результати такого застосування;
 - знати особливості реалізації Шаблону, споріднені шаблони, відомі випадки його застосування в програмних додатках;
 - вільно володіти структурою Шаблону, призначенням його класів та відносинами між ними;
 - вміти розпізнавати Шаблон в UML діаграмі класів та будувати сирцеві коди Java-класів, що реалізують шаблон.
3. В підготованому проєкті створити програмний пакет work6. В пакеті розробити інтерфейси і класи, що реалізують завдання 1 та 2 (згідно варіанту) з застосуванням одного чи декількох шаблонів (п.2). В класах, що розробляються, повністю реалізувати методи, пов'язані з функціонуванням Шаблону. Методи, що реалізують бізнес-логіку, закрити заглушками з виводом на консоль інформації про викликаний метод та його аргументи. Приклад реалізації бізнес-методу: `void draw(int x, int y) {System.out.println("Метод draw з параметрами x="+x+" y="+y);}`
4. За допомогою автоматизованих засобів виконати повне документування розроблених класів (також методів і полів), при

цьому документація має в достатній мірі висвітлювати роль певного класу в загальній структурі Шаблону та особливості конкретної реалізації.

Визначимо варіант для індивідуальних завдань: $4214\%7 = 0$

Варіант 0 до завдання 1:

- **Необхідно створити модель для представлення композитної структури ігрового простору**, яка включає такі елементи:
 - **GameElement (Елемент гри)** – абстрактний клас або інтерфейс, що описує загальні властивості елементів і метод clone(), який реалізує клонування (глибоке або поверхневе).
 - Клас, який може містити інші елементи (GameElement) і делегує виконання дій усім своїм піделементам. Приклад: кімнати, рівні, групи об'єктів.
 - Клас, який представляє конкретний елемент, що не містить піделементів. Приклад: персонажі, предмети, перешкоди.
 - Інтерфейс або абстрактний клас, що визначає метод clone(), який дозволяє створювати копії об'єктів.
- **Вимоги:**
 - Створити абстрактний клас або інтерфейс GameElement з методом clone().
 - При клонуванні:
 - ♦ **Поверхневе клонування** – копіюється лише посилання на піделементи.
 - ♦ **Глибоке клонування** – створюється повна копія всіх піделементів, включно з їхніми власними піделементами.
- У методі main продемонструвати:
 - Створення структури ігрового простору (наприклад, рівень з кількома кімнатами та об'єктами).
 - Виконання поверхневого та глибокого клонування структури.
 - Показати, що зміни в глибокій копії не впливають на оригінал, а при поверхневій копії – впливають.

Код до завдання 1:

Вміст файлу Main.java:

```
package work6;

public class Main {
    public static void main(String[] args) {
        GameGroup castle = new GameGroup("[Level 1]");
        GameGroup tower = new GameGroup("Sleep Princess tower");
        GameUnit hero = new GameUnit("new 'King'");
        GameUnit boss = new GameUnit("Dragon (should take care about princess)");
        GameUnit princess = new GameUnit("Sleepy (Really) Princess");

        tower.add(hero);
        tower.add(boss);
        tower.add(princess);
        castle.add(tower);

        System.out.println("[ORIGINAL] ---");
        castle.display("");

        GameGroup shallow = (GameGroup) castle.clone();
        GameGroup deep = (GameGroup) castle.deepClone();

        System.out.println("\n>> new 'King' kill Dragon...");
        hero.setName("new (now really??) King");
        boss.setName("(Already) killed Dragon");
        princess.setName("Unslepy (but not kissed!) princess");
        System.out.println("\n[Copy result] ---");
        shallow.display("");
        System.out.println("\n[Deep copy result] ---");
        deep.display("");
    }
}
```

Вміст файлу GameElement.java:

```
package work6;

/**
 * Абстрактний базовий клас для всіх об'єктів ігрового простору.
 * Реалізує патерн Composite (предок для груп) і Prototype (дозволяє копіювати)
 */
public abstract class GameElement {
    /**
     * Назва ігрового об'єкта
     */
    protected String name;

    /**
     * Конструктор для створення базового ігрового елемента
     * @param name початкова назва елемента
     */
    public GameElement(String name) {
        this.name = name;
    }

    /**
     * Метод виведення імені для ігрового елемента
     */
}
```

```

    * @return рядок з назвою
    */
    public String getName() {
        return name;
    }

    /**
     * Метод встановлення імені для ігрового елементу
     * @param name нова назва
     */
    public void setName(String name) {
        this.name = name;
    }

    /**
     * Метод виведення інформації про частину
     * @param indent рядок відступу для гарного форматування дерева (наприклад, " ")
     */
    public abstract void display(String indent);

    /**
     * Метод поверхневого копіювання
     * @return новий об'єкт, що є поверхневою копією поточного
     */
    public abstract GameElement clone();

    /**
     * Метод глибокого копіювання
     * @return новий об'єкт, що є повною незалежною копією поточного
     */
    public abstract GameElement deepClone();
}

```

Вміст файлу GameUnit.java:

```

package work6;
/**
 * Листовий елемент ігрового простору.
 */
public class GameUnit extends GameElement{
    public GameUnit(String name)
    { super(name); }

    @Override
    public void display(String indent) {
        System.out.println(indent + "| Unit | " + name);
    }

    @Override
    public GameElement clone() {
        return new GameUnit(this.name);
    }

    @Override
    public GameElement deepClone() {
        return new GameUnit(this.name);
    }
}

```

Вміст файлу GameGroup.java:

```

package work6;

import java.util.ArrayList;
import java.util.List;

/**
 * Композитний елемент ігрового простору (Composite).
 */
public class GameGroup extends GameElement {
    /** Список дочірніх елементів, що входять до цієї групи */
    private List<GameElement> children = new ArrayList<>();

    public GameGroup(String name) {
        super(name);
    }
    /**
     * Додає новий елемент до складу групи
     * @param gameElement об'єкт, який треба додати
     */
    public void add(GameElement gameElement) {
        children.add(gameElement);
    }

    /**
     * Виводить назву групи усіх нащадків
     * @param indent поточний відступ.
     */
    @Override
    public void display(String indent) {
        System.out.println(indent + "[Group]: " + name);
        for (GameElement child : children) child.display(indent + " ");
    }

    @Override
    public GameElement clone() {
        GameGroup copy = new GameGroup(this.name);
        for (GameElement child : this.children) copy.add(child);
        return copy;
    }

    @Override
    public GameElement deepClone() {
        GameGroup copy = new GameGroup(this.name);
        for (GameElement child : this.children) copy.add(child.deepClone());
        return copy;
    }
}

```

Результат виконання програми:

```
[ORIGINAL] ---
+ [Group]: [Level 1]
+ [Group]: Sleep Princess tower
  | Unit | new 'King'
  | Unit | Dragon (should take care about princess)
  | Unit | Sleepy (Really) Princess

>> new 'King' kill Dragon...

[Copy result] ---
+ [Group]: [Level 1]
+ [Group]: Sleep Princess tower
  | Unit | new (now really??) King
  | Unit | (Already) killed Dragon
  | Unit | Unslepy (but not kissed!) princess

[Deep copy result] ---
+ [Group]: [Level 1]
+ [Group]: Sleep Princess tower
  | Unit | new 'King'
  | Unit | Dragon (should take care about princess)
  | Unit | Sleepy (Really) Princess
```

Рисунок 1 – результат виконання програми згідно першого завдання

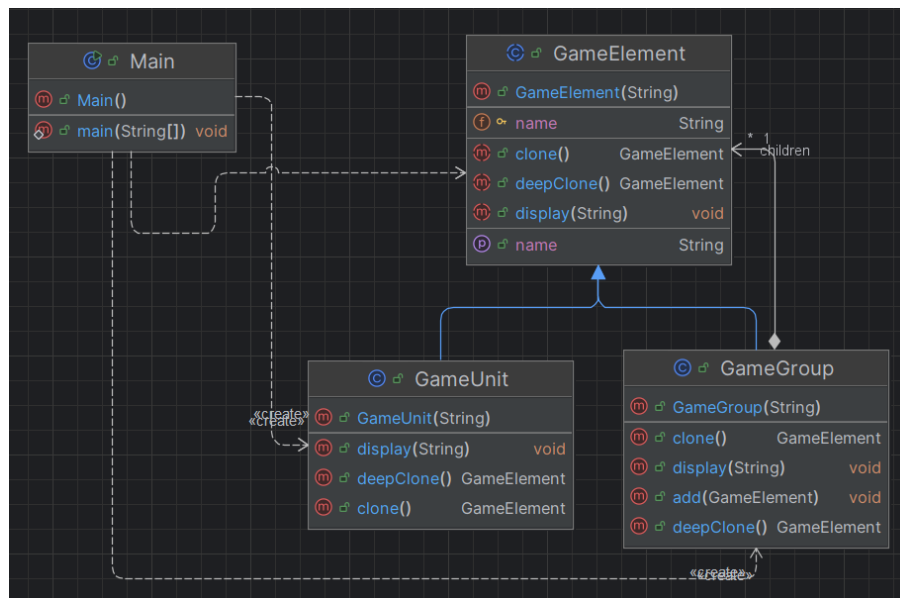


Рисунок 2 – Побудована в IntelliJ IDEA UML діаграма для першого завдання

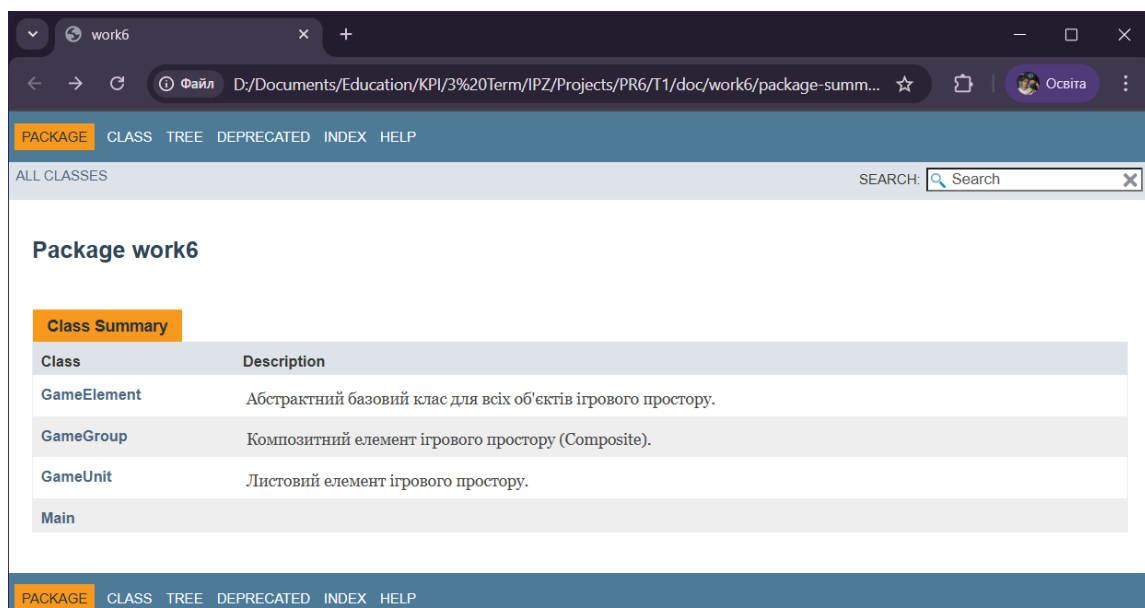


Рисунок 3 – Створена документація у форматі веб сторінки

Варіант 0 до завдання 2:

- Визначити специфікації класів, які інкапсулюють лінійний список цілих чисел та ітератор послідовного обходу у прямому напрямку в упорядкованій структурі.
- Вимоги:
 - Створити інтерфейс або абстрактний клас `IntegerList`, який містить методи для додавання, видалення та отримання елементів. Підтримує збереження елементів у впорядкованому вигляді (наприклад, за зростанням). Метод `createIterator()` – фабричний метод, що повертає об'єкт ітератора.
 - Клас `OrderedIntegerList` – конкретна реалізація `IntegerList`, який додає елементи так, щоб зберігався порядок. Містить метод `createIterator()`, який повертає ітератор для послідовного обходу списку.
 - Інтерфейс або абстрактний клас `Iterator` містить методи `hasNext()` та `next()` для обходу елементів списку у прямому напрямку.
 - Клас `OrderedIterator` – конкретний ітератор для `OrderedIntegerList`, який реалізує методи `hasNext()` та `next()`. Забезпечує послідовний обхід елементів від початку до кінця списку.
- У методі `main` продемонструвати:
 - Створити список.
 - Додати кілька чисел до списку.
 - Викликати `createIterator()` для отримання ітератора.
 - Зробити послідовний обхід елементів за допомогою `hasNext()` та `next()`.
 - Вивести елементи у консоль, перевірити правильності впорядкування.

Код до завдання 2:

Вміст файлу Main.java:

```
import work6.*;
public class Main {
    public static void main(String[] args) {
        IntegerList list = new OrderedIntegerList();
        list.add(10);
        list.add(5);
        list.add(20);
        Iterator it = list.createIterator();
        System.out.println("[!] Iteration cycle:");
        while (it.hasNext()) {
            System.out.print(it.next() + " ");
        }
    }
}
```


Вміст файлу Iterator.java:

```
package work6;

/**
 * Інтерфейс продукту, який створюється фабричним методом
 */
public interface Iterator {
    /** Метод перевірки наявності наступного елемента */
    boolean hasNext();
    /** Метод виведення наступного елемента */
    Integer next();
}
```

Вміст файлу OrderedIterator.java:

```
package work6;

/**
 * Реалізація продукту, яка вміє обходити конкретний список
 */
public class OrderedIterator implements Iterator {
    private OrderedIntegerList list;
    private int index = 0;

    /** Базовий конструктор
     * @param list заданий список
     */
    public OrderedIterator(OrderedIntegerList list) {
        this.list = list;
    }

    @Override
    public boolean hasNext() {
        return index < list.size();
    }

    @Override
    public Integer next() {
        if (!hasNext()) return null;
        return list.get(index++);
    }
}
```

Вміст файлу IntegerList.java:

```
package work6;

/**
 * Інтерфейс оголошення фабричного методу createIterator()
 */
public interface IntegerList {
    /** Метод для додавання елементів у список */
    void add(int integer);
    /** Метод для видалення елемента з списку */
    void remove(int integer);
    /** Метод, який повертає елемент за індексом */
    int get(int index);
    /** Метод, який потрібен для сортування */
    int size();
    /** Фабричний метод, що повертає об'єкт ітератора */
    Iterator createIterator();
}
```

Вміст файлу OrderedIntegerList.java:

```
package work6;
import java.util.ArrayList;
import java.util.List;

public class OrderedIntegerList implements IntegerList {
    private final List<Integer> data;

    /** Базовий конструктор створення списку */
    public OrderedIntegerList() {
        data = new ArrayList<>();
    }

    @Override
    public void add(int value) {
        int i = 0;
        while (i < data.size() && data.get(i) < value) {
            i++;
        }
        data.add(i, value);
    }

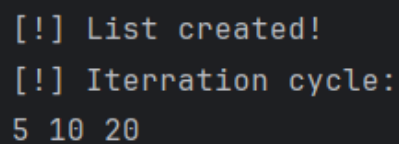
    @Override
    public void remove(int value) {
        data.remove((Integer) value);
    }

    @Override
    public int size() {
        return data.size();
    }

    @Override
    public int get(int index) {
        if (index >= 0 && index < data.size()) {
            return data.get(index);
        }
        System.out.println("Index out of bounds");
        return -1;
    }

    @Override
    public Iterator createIterator() {
        System.out.println("[!] List created!");
        return new OrderedIterator(this);
    }
}
```

Результат виконання програми:



```
[!] List created!
[!] Iterration cycle:
5 10 20
```

Рисунок 4 – результат виконання програми згідно другого завдання

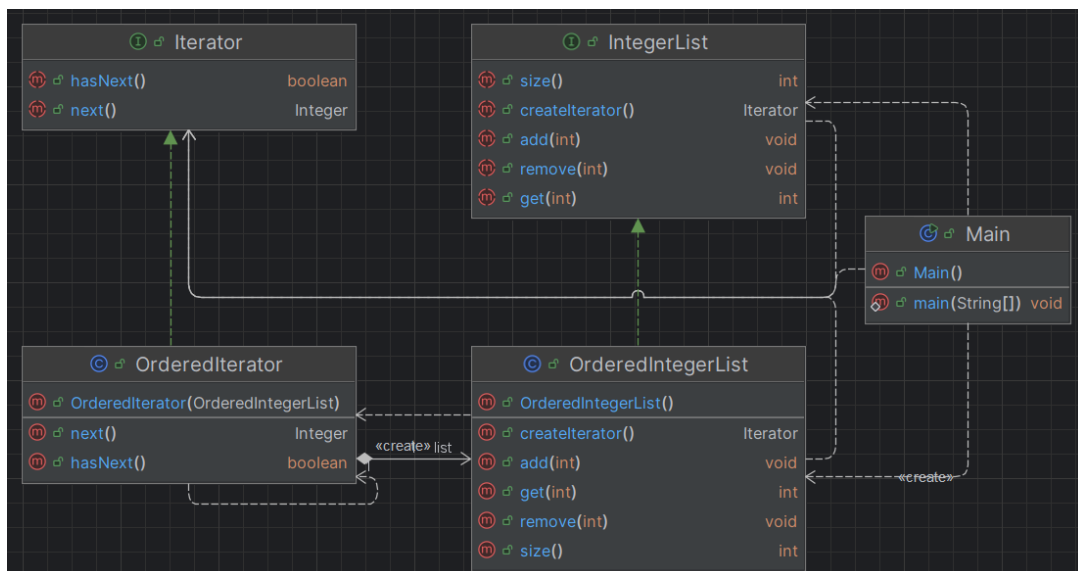


Рисунок 5 – Побудована в IntelliJ IDEA UML діаграма для другого завдання

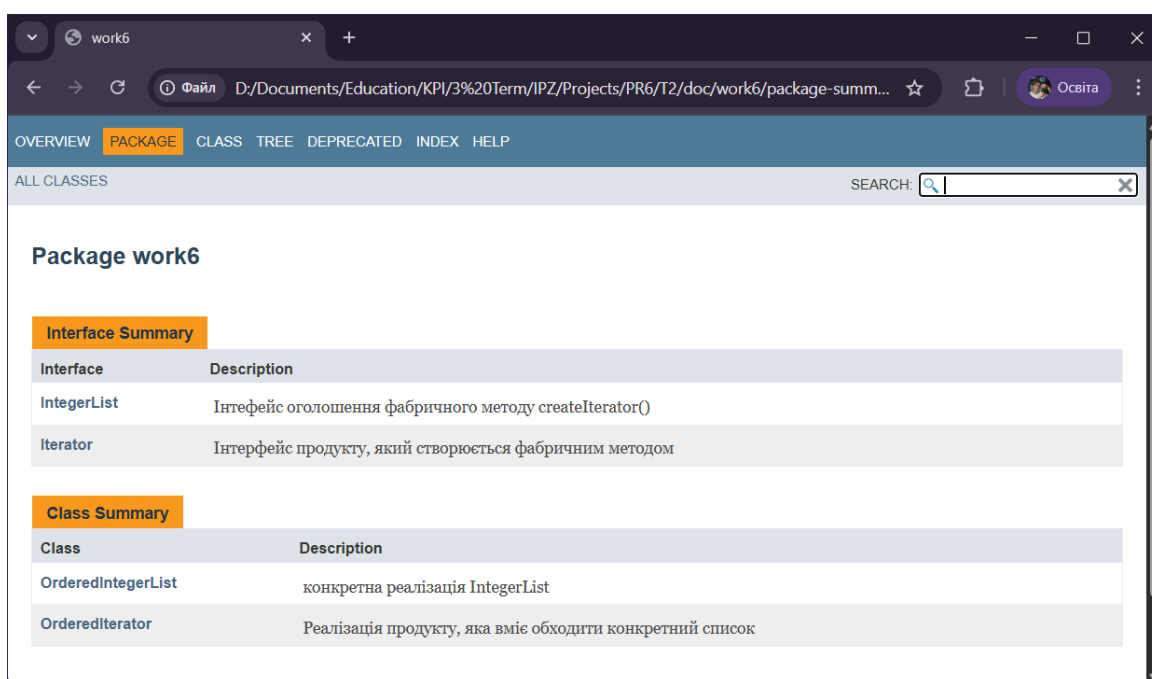


Рисунок 6 – Створена документація у форматі веб сторінки

Висновки:

У першому завданні створено модель ігрового простору, що дозволяє виконувати глибоке та поверхнєве копіювання об'єктів за допомогою шаблону Prototype (у поєднанні з Composite). У другому завданні розроблено впорядкований список, де застосовано шаблон Factory Method для створення об'єкта Iterator, що забезпечує послідовний обхід елементів колекції без розкриття її внутрішньої реалізації. Увесь код міститься на даному GitHub репозиторії: <https://github.com/BronievM/KPI-IPZ-2025/tree/main/PR6>